Block size: 64 - 128 - 256 - 512

↳ Only small differences; 512 was slightly faster

=> enough threads were available at every size to saturate memory

Array Size: 2^{10} - 2^{26}

↳ small: ~6 GB/s; large: ~249 GB/s

=> small arrays are dominated by launch overhead; large arrays expose memory bandwidth

Memory access: Contiguous vs. stride 4

↳ stride 4 was ~5.5x slower

=> neighboring threads should access neighboring memory

```
Contiguous:
Threads:  0  1  2  3  4
Memory:   0  1  2  3  4    → packed/coalesced transactions

Stride 4:
Threads:  0  1  2  3  4
Memory:   0  4  8 12 16    → scattered, more transactions
```

Data Type: fp32 vs. fp16

↳ fp16 took half the time, but both reached ~249 GB/s

=> fp16 moves half as many bytes; confirms memory-bound behavior

Element/thread: 1 vs 4

↳ 4 elements/thread was ~2% slower

=> fewer threads did not reduce the main cost: memory traffic

float4: scalar vs. vectorized access

└ float4 was slightly slower

=> wider instructions do not necessarily help an already bandwidth-bound kernel

grid-stride: full grid vs. reusable fixed grid

└ slightly slower than basic

=> more flexible for large workloads, but has loop/index overhead

Main conclusion

> Vector addition performs very little computation but transfers lots of data. Performance therefore depends mainly on memory bandwidth and coalesced access—not on reducing arithmetic instructions or giving each thread more work.